

# SmartKnob firmware and software

<https://github.com/scottbez1/smartknob>

 GitHub - scottbez1/smartknob: Haptic input knob with software-defined endstops and virtual detents • githu...

Comments or questions are welcome! (Dropbox account required to sign into Paper)

Or, join the [Discord](#) for more lively discussion.

## Dev Environment

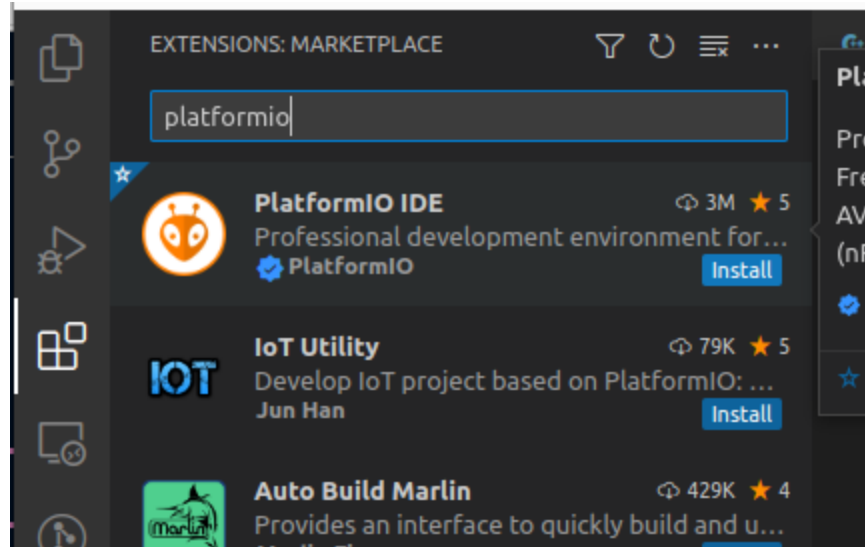
SmartKnob uses PlatformIO to build the firmware. The recommended development environment is VS Code. Although the firmware uses some Arduino libraries/framework code, it cannot be developed using the Arduino IDE.

## Getting started

💡 Before we start, let's set some expectations: the firmware currently only provides a basic framework for experimentation that allows haptic detent configuration and serial input/output as a demo, but doesn't support any "real world" usage out of the box (i.e. there's no MQTT, home assistant, macropad, or other features implemented — but reach out in Discord if you'd like to help contribute!).

## Install PlatformIO extension in VS Code

From the extensions sidebar, install PlatformIO IDE

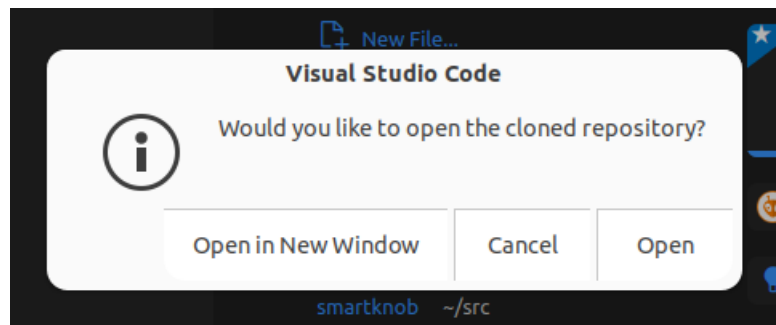
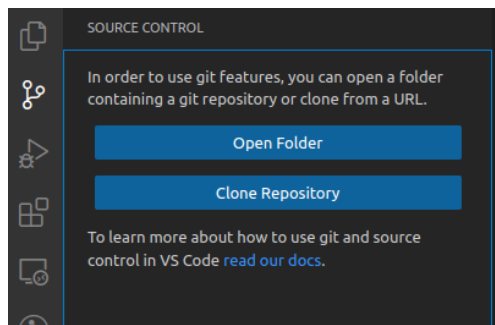


## Open the source code

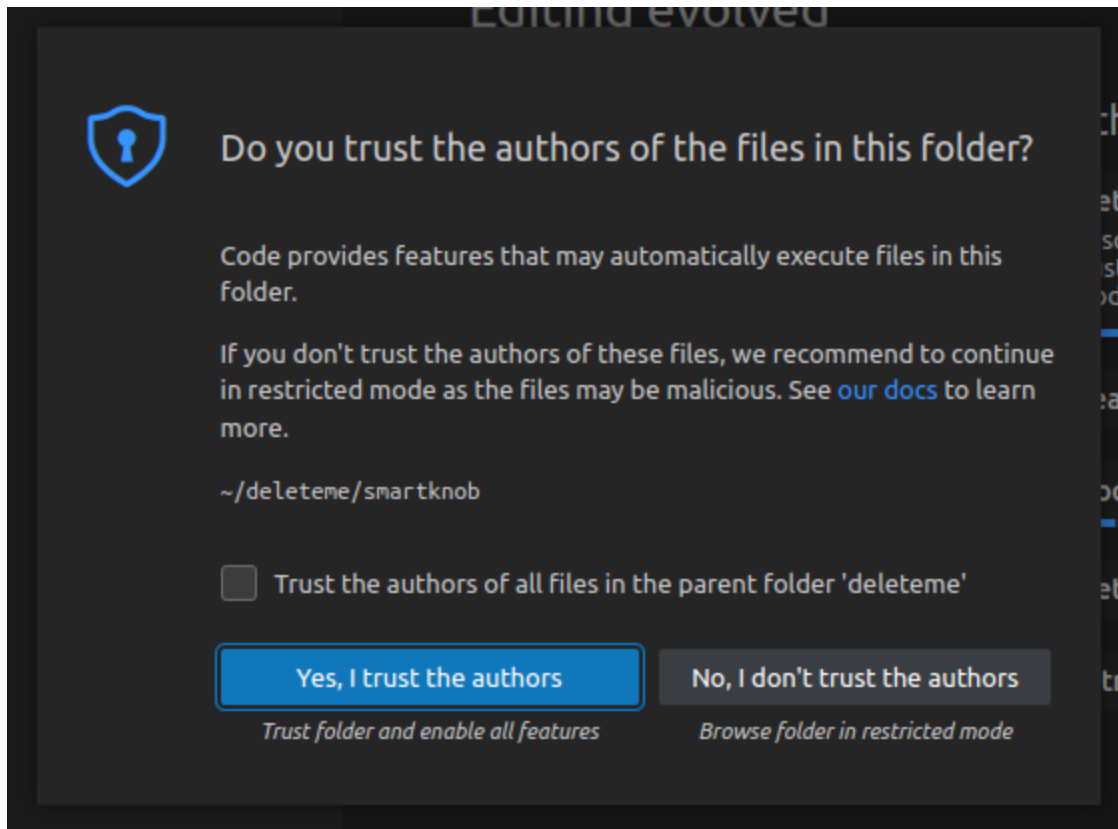
Use File→Open Folder, and select the root of the smartknob git repo.

If you haven't already cloned the git repo, select the "Source Control" sidebar, "Clone Repository" and enter `https://github.com/scottbez1/smartknob.git` as the repository url.

Select a location on your computer to store the code, Open it

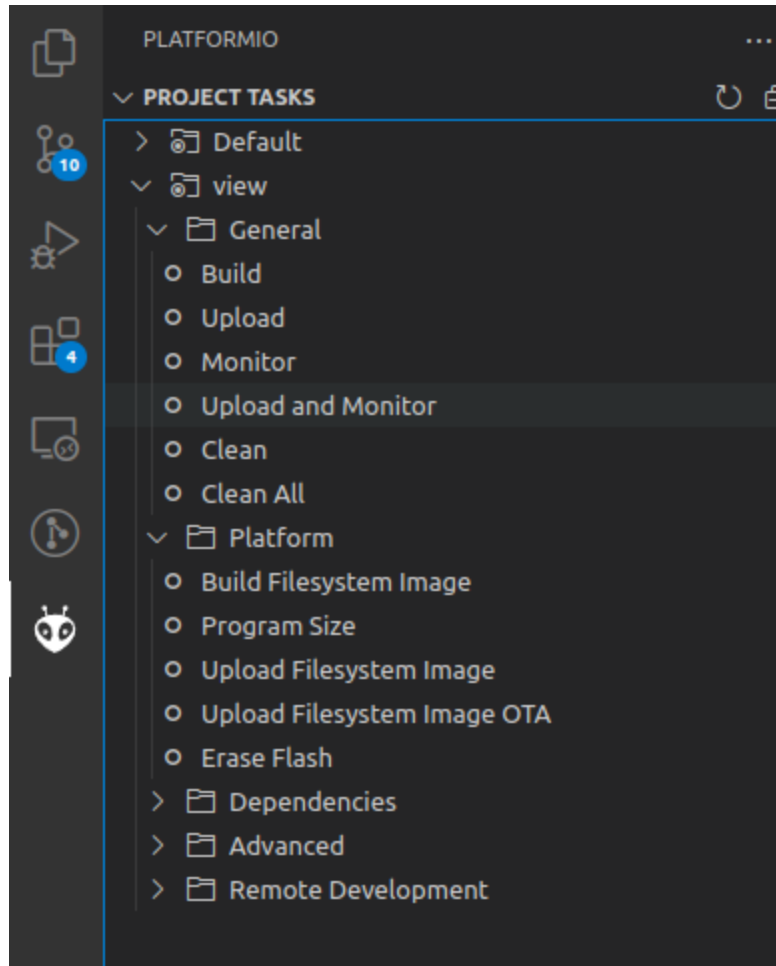


Select "Yes, I trust the authors" (required for the PlatformIO extension to be able to run/compile code)



## Upload firmware

From the PlatformIO sidebar, select "Upload and Monitor"



You should see a boot up message after upload.

```
PROBLEMS 20 OUTPUT DEBUG CONSOLE TERMINAL GIT LENS SEARCH

Writing at 0x00018f14... (15 %)
Writing at 0x0002a7da... (23 %)
Writing at 0x0003da2a... (30 %)
Writing at 0x0004372c... (38 %)
Writing at 0x00048da9... (46 %)
Writing at 0x0004e0f2... (53 %)
Writing at 0x00055457... (61 %)
Writing at 0x0005d893... (69 %)
Writing at 0x00063518... (76 %)
Writing at 0x0006966e... (84 %)
Writing at 0x0006f4ef... (92 %)
Writing at 0x00074f6f... (100 %)
Wrote 422288 bytes (201784 compressed) at 0x00010000 in 5.0 seconds (effective 676.1 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
===== [SUCCESS] Took 12.92 seconds =====
--- Terminal on /dev/ttyUSB0 | 921600 8-N-1
--- Available filters and text transformations: colorize, debug, default, direct, esp32_exception_decoder, hexlify, log2file, nocontrol, printable, send_on_enter, ti
me
--- More details at https://bit.ly/pio-monitor-filters
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
SmartKnob starting!

Serial mode: plaintext
Press 'C' at any time to calibrate.
Press <Space> to change haptic modes.
LOG: Sprite created!
LOG: millilux: 24948.00
LOG: HX711 reading: 1065840
```

# Calibration

## Motor calibration

Motor calibration is REQUIRED for any haptic feedback to work correctly. It tunes the encoder to your specific motor's magnet and coil orientation. To run calibration, type capital 'C' into the serial terminal at any time and then wait for the calibration procedure to complete — don't touch the knob while calibration is running. The motor should spin smoothly for a bit, followed by a bunch of jerky movements in both directions. The calibration should be automatically saved to internal flash memory — look for `Wrote <X> bytes` and `Success!` messages

## Strain calibration

The SmartKnob View uses analog strain gauges to measure presses on the knob. You'll need to calibrate the thresholds based on the specific values you get from your strain gauges once assembled. To run calibration, type capital `S` into the serial terminal at any time and follow the prompts to complete calibration. Once this is done you should be able to press down on the knob to switch through demo modes, and the LEDs should react to the amount of pressure you apply.

You can find some additional configuration flags in the `platformio.ini` file, e.g. if you'd like to disable the strain gauge pressure sensitivity, or disable the VEML7700 ambient light sensor.

# Modifying Firmware - Architecture Guide

If you're interested in tweaking the firmware, start by reading through this quick high-level guide to the existing firmware to understand some of the key concepts, then I'd definitely recommend poking around the code and joining the discord to discuss!

## Concepts

### Config vs state

Config and State represent the key inputs/outputs to the SmartKnob.

"Config" is the specification of the desired configuration of the device. For example, the number of detents, or the strength/torque of detents. Config is *pushed into* the

SmartKnob.

Config is idempotent (that is, applying the same config multiple times in a row doesn't change the result). Config is always specified from outside the system, and shouldn't internally change in response to the state of the system (however, an external controller *may* decide to change the configuration in response to changing state). [caveat: as of 2023-02-10, out of laziness and bad coding, the current position state is actually stored in a `config` data structure in `motor_task.cpp`. This should be fixed]

"State" is the representation of observable aspects of the system. For example, the current detent position (or sub-position). State is *published by* the SmartKnob.

### Haptic config

Currently there is a single haptic controller, which is configurable for a wide variety of behaviors using a handful of parameters. These are documented in the [SmartknobConfig proto definition](#) and example values for the demo modes can be found in the [interface\\_task demo implementations](#).

## Tasks

The firmware takes advantage of FreeRTOS tasks (essentially "threads" if you're familiar with multi-threaded software) and both cores of the ESP32. One core is dedicated to the motor control loop, which is why the firmware may not work on a single-core ESP32 device (e.g. ESP32-S2).

Tasks communicate with each other using FreeRTOS queues.

### `motor_task.cpp`

Runs the SimpleFOC motor control loop and provides haptic feedback through the motor. This needs to run at a high speed to ensure a responsive/stable control system, so it's the only task assigned to core 1.

This file contains motor calibration constants at the top which correspond to the physical angular alignment of the motor's electrical phase relative to the angle sensed by the magnetic encoder — these need to be tuned for *your specific motor and hardware assembly* (see [Calibration](#) in the Getting Started section above)

### **interface\_task.cpp**

This task is responsible for general input/output of the SmartKnob (other than the main haptics), which includes:

- Serial communication (defaults to plaintext mode, but supports a protobuf-based mode for programmatic access, see [Serial](#) below)
- Strain gauge pressure/click detection
- Ambient light sensing and backlight/LED-ring control
- A “demo” controller that connects the strain-gauge clicks (or input from serial) to switch through haptic “modes”; this is the same demo seen in the original video

### **display\_task.cpp**

Drives the display using TFT\_eSPI directly. (There’s an lvgl branch in the works for theoretically higher fidelity graphics; help wanted to improve it, particularly performance!)

This task registers a queue with the motor task so it waits for state updates to be posted to that queue to re-render the screen.

**mqtt\_task.cpp** (only on the `dev/mqtt` branch as of 2023-02-10)

Work-in-progress. Manages wifi and mqtt connection. As of 2023-02-10, it just connects to a server (specified in secrets.h), listens to a single topic, and ignores all messages received.

Unclear what the end-goal looks like here. Likely will have distinct topics for config vs state. Protobuf messages would be preferred (see [Serial](#) section below), but JSON seems like an unfortunate likelihood given the rest of the MQTT ecosystem.

## **Serial**

In order for a computer to communicate with the SmartKnob, it appears as a USB serial device.

However, usage of Arduino’s `Serial` is strictly forbidden, and instead a `logger` abstraction is provided for sending basic text debug logs. Other data is transferred in a structured way, described below.

This allows flexibility in the format of data transferred over serial, and in fact the SmartKnob provides 2 different serial modes that serve different purposes.

## Plaintext mode

By default, it starts in “plaintext” mode, which is developer-friendly and you’re probably familiar with if you’ve opened a serial monitor with the SmartKnob connected:

```
--- Terminal on /dev/ttyUSB0 | 921600 8-N-1
SmartKnob starting!

Serial mode: plaintext
Press 'C' at any time to calibrate.
Press <Space> to change haptic modes.
LOG: Sprite created!
LOG: HX711 reading: 1072564
LOG: millilux: 15494.40
STATE: 0 [0, -1] (detent strength: 0.00, width: 10 deg, endst
op strength: 1.00)
LOG: adjusting detent center
LOG: Got new config
LOG: millilux: 15458.40
LOG: HX711 reading: 1072297
STATE: 2 [0, -1] (detent strength: 0.00, width: 10 deg, endst
op strength: 1.00)
STATE: 3 [0, -1] (detent strength: 0.00, width: 10 deg, endst
op strength: 1.00)
STATE: 4 [0, -1] (detent strength: 0.00, width: 10 deg, endst
op strength: 1.00)
STATE: 6 [0, -1] (detent strength: 0.00, width: 10 deg, endst
op strength: 1.00)
```



However, this isn't great for programmatically configuring or receiving updates from the SmartKnob, so instead the firmware offers a programmatic interface using a binary protocol based on Google's [Protobuf](#) standard.

## Protobuf (binary/programmatic) mode

The protobuf-based binary serial mode is a compact and flexible way to transfer structured data from the host computer to the SmartKnob and vice-versa.

### Benefits of protobuf

protobuf provides several benefits over other encoding mechanisms like JSON:

1. Well-defined schema. If you're curious about the format of data to expect or send, you just need to check the [protobuf file](#)
2. Code generation. Instead of hand-writing JSON parsers every time the data changes, protobuf provides code generation of the encoding and decoding logic, and data-structures. SmartKnob uses nanopb to generate C structs based off the schema, and all the code for encoding/decoding that data from the binary format.
3. Relatively compact/efficient wire encoding. It's not a primary goal in this project, but the binary wire encoding is generally fairly compact, due to omitting default/unspecified fields, using variable length encodings, etc. It's certainly much more compact than JSON which uses strings to describe every field in every message.
4. Backwards/forwards compatibility. Not super relevant to this project, but many common schema changes are backwards and forwards compatible, meaning an older client or a newer client will be able to handle them gracefully.

### Disadvantages of protobuf

1. Not human-readable. This makes it much harder to debug, as you need something that can interpret messages. Since they're binary, viewing anything in a terminal directly will be fruitless.
2. Not self-describing. If you come across a JSON document, you can generally tell what it means because fields have string names/labels describing their contents. Protobuf has no such thing (it uses integer field numbers, which does make renaming fields easier) so you need to have a copy of the schema (.proto file - and

you'd better hope it matches the data!) in order to understand an encoded message.

This is why the SmartKnob defaults to plaintext mode to make basic validation/debugging easier.

### **How it works**

Protobuf messages are encoded to their binary wire format and a CRC32 checksum appended. Then that entire binary string is COBS encoded into a packet, and delimited/framed by 0 (NULL) bytes when sent over serial. This provides a basic packet-based interface with integrity checks (rather than the raw, stream-based interface of a serial connection).

The SmartKnob automatically switches to binary protobuf mode when it receives a 0 byte.

## **Software**

Software running on a Host computer can configure the SmartKnob's virtual detents and react to rotation/interaction from the knob. You could implement software that serves as the glue to connect the SmartKnob to your favorite application (e.g. using Adobe Premier's SDK). The sky's the limit.

Due to the protobuf-based protocol, it's relatively straightforward to interact with the SmartKnob from any language that has a protobuf compiler (like Python, Typescript, Go, Kotlin, Java, C++, etc)

Example libraries and demo applications for Python and Typescript (Node.js and browser via Web Serial) are provided.

If you want to write your own library in another language, you'll need to replicate some basic features of the existing libraries:

1. COBS decoding - to receive a message from the SmartKnob, the byte stream needs to be broken into 0 (NULL) delimited packets
2. CRC verification - Once a packet has been extracted, its CRC32 should be validated to confirm the data was not corrupted

The inverse is true for messages sent TO the SmartKnob - you'll need to add CRC's and COBS encode them with 0 (NULL) delimiters when sending them.

When sending "config" to the SmartKnob, the library should internally use a queue and implement retry logic (since config is idempotent) to provide reliable config delivery. Currently this requires full round-trip Ack messages to confirm receipt of each message sent to the SmartKnob, but config changes should be somewhat infrequent, so performance is not a huge concern.

Each unique message queued should be assigned a unique nonce, which will be echoed back in an Ack message for confirmation of receipt. The first message sent by the library should always have a **random** nonce over the full 32-bit space; if you use a consistent starting nonce every time the app starts (like 0) it's possible the SmartKnob will ignore that first message, which may be confusing.